

Linked List Pattern Notes by Aman

1. The Fast and Slow Pointer (Tortoise and Hare)

This is the "bread and butter" of Linked List problems. You maintain two pointers moving at different speeds.

- **Why use it:** To find a specific position or detect a cycle without storing extra data (like a HashSet).
- **How to Identify:**
 - Find the middle of the list.
 - Detect if a cycle exists.
 - Find the k^{th} node from the end.
 - Find the entry point of a cycle.

Java

```
ListNode slow = head;
ListNode fast = head;

while (fast != null && fast.next != null) {
    slow = slow.next;          // moves 1 step
    fast = fast.next.next;     // moves 2 steps

    if (slow == fast) {
        // Cycle detected logic
    }
}

// After loop, slow is at the middle (for non-cyclic)
```

2. The Reverse Linkage Pattern

Many problems require you to manipulate the direction of the `next` pointers rather than moving the nodes themselves.

- **Why use it:** To reverse the list in-place ($O(1)$ space).
- **How to Identify:**
 - "Reverse the list."
 - "Check if a list is a Palindrome" (Reverse the second half).
 - "Reverse nodes in k-group."

Java

```
ListNode prev = null;
ListNode curr = head;

while (curr != null) {
    ListNode nextTemp = curr.next; // store next
    curr.next = prev;              // reverse link
    prev = curr;                   // move prev forward
    curr = nextTemp;               // move curr forward
}
return prev; // New head
```

3. The Dummy Node (Sentinel)

A dummy node is a placeholder node placed at the beginning of the list.

- **Why use it:** It simplifies edge cases, specifically when the **head** of the list might be removed or changed. It prevents `NullPointerException`.
- **How to Identify:**
 - Merging two sorted lists.
 - Removing a node (where the head might be the target).
 - Partitioning a list around a value.

Java Template:

Java

```
ListNode dummy = new ListNode(0);
ListNode curr = dummy;

while (condition) {
    if (someLogic) {
        curr.next = newNode;
        curr = curr.next;
    }
}

return dummy.next; // The actual head
```

4. Two-Pointer (Gap/Fixed Distance)

Unlike Fast and Slow, these pointers move at the **same speed**, but one starts later than the other.

- **Why use it:** To find a node relative to the end of the list in a single pass.
- **How to Identify:**
 - "Remove the N^{th} node from the end."
 - "Find the intersection of two lists."

Java Template:

Java



```
ListNode first = head;
ListNode second = head;

// Create a gap of N
for (int i = 0; i < n; i++) first = first.next;

// Move both until first hits the end
while (first != null) {
    first = first.next;
    second = second.next;
}
// 'second' is now at the target position
```

5. In-Place Reordering

This is a "composite pattern" that combines several techniques.

- **Why use it:** To transform the list structure without creating new nodes.
- **How to Identify:**
 - "Reorder List" (e.g., $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \dots$)
 - "Zig-Zag" patterns.
- **The Strategy:**
 1. **Find the middle** (Fast/Slow Pointer).
 2. **Reverse the second half** (Reverse Pattern).
 3. **Merge the two halves** (Two-Pointer/Dummy Node).

6. The "Interweaving" Pattern (Copying with Random Pointers)

This is a highly specific pattern used when a node has a `next` pointer and a `random` pointer that could point anywhere in the list.

- **Why use it:** To clone a complex list in $O(N)$ time with $O(1)$ extra space (excluding the new list).
- **How to Identify:** "Copy List with Random Pointer" or any problem involving deep-copying a non-linear linked structure.
- **The Strategy:**
 1. **Iterate** and create a copy of each node, inserting it directly after the original ($A \rightarrow A' \rightarrow B \rightarrow B'$).
 2. **Assign random pointers:** `curr.next.random = curr.random.next`.
 3. **Separate the lists:** Restore original links and extract the copy.

7. The "K-Way Merge" (Heap + Linked List)

This pattern applies when you aren't just merging two lists, but many.

- **Why use it:** To efficiently combine K sorted lists into one sorted list.
- **How to Identify:** "Merge K Sorted Lists."
- **Java Template:** Using a **PriorityQueue** (Min-Heap) to keep track of the smallest current node across all lists.

Java



```
public ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> a.val - b.val);
    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;

    // Add the head of each list to the heap
    for (ListNode node : lists) {
        if (node != null) pq.add(node);
    }

    while (!pq.isEmpty()) {
        ListNode smallest = pq.poll();
        curr.next = smallest;
        curr = curr.next;

        // If the polled node has a next, add it to the heap
        if (smallest.next != null) pq.add(smallest.next);
    }
    return dummy.next;
}
```

8. Recursive Traversal (The "Backwards" Pattern)

Linked lists are naturally recursive structures. While iteration is usually preferred for space ($O(1)$), recursion allows you to process nodes **post-order** (from tail to head).

- **Why use it:** When you need information from the end of the list to make a decision at the beginning.
- **How to Identify:** * "Print/Process list in reverse."
 - "Add two numbers represented by linked lists" (where you must align the digits from the end).
 - "Swap nodes in pairs" (cleaner code with recursion).

Java Template:

Java



```
public void recurse(ListNode node) {  
    if (node == null) return;  
  
    // Logic here would be PRE-ORDER (Head to Tail)  
  
    recurse(node.next);  
  
    // Logic here happens POST-ORDER (Tail to Head)  
    // The stack "remembers" the previous nodes for you.  
}
```

The Linked List Pattern Map

Pattern	Famous Problems	Key Difficulty
Fast & Slow Pointer	Linked List Cycle (#141), Middle of the Linked List (#876), Happy Number (#202)	Easy/Medium
Reverse Linkage	Reverse Linked List (#206), Palindrome Linked List (#234)	Easy/Medium
Dummy Node	Merge Two Sorted Lists (#21), Remove Nth Node From End (#19), Partition List (#86)	Medium
Two-Pointer (Gap)	Intersection of Two Linked Lists (#160), Remove Nth Node From End (#19)	Medium
In-Place Reordering	Reorder List (#143), Reverse Nodes in k-Group (#25)	Hard
Interweaving	Copy List with Random Pointer (#138)	Medium/Hard
K-Way Merge	Merge k Sorted Lists (#23)	Hard
Recursive Traversal	Reverse Linked List (#206), Add Two Numbers (#2)	Medium

MERGE K Sorted Linked list

Input:- [

1 → 4 → 5,

1 → 3 → 4,

2 → 6

]

merging items :- 1 → 1 → 2 → 3 → 4 → 4 → 5 → 6 → null

⇒ My first approach would be

Sent every two list which merge two sorted list
then that sorted list sent again with another sorted list
this thing work until all list empty.

```
Node helper(Node[] arr, int i, int k, Node merge)
{
    if (i == k)
        return merge;
}
```

③

②

Now this code have time

Complexity = $O(NK^2)$

```
Node merge = (Node) arr[i+1];
return helper(arr, i+1, k, merge);
```

Optimized approach :- $O(NK \log k)$

* use divide and conquer rule

```
Node helper(Node[] arr, int si, int ei)
```

```
{
    if (si == ei)
```

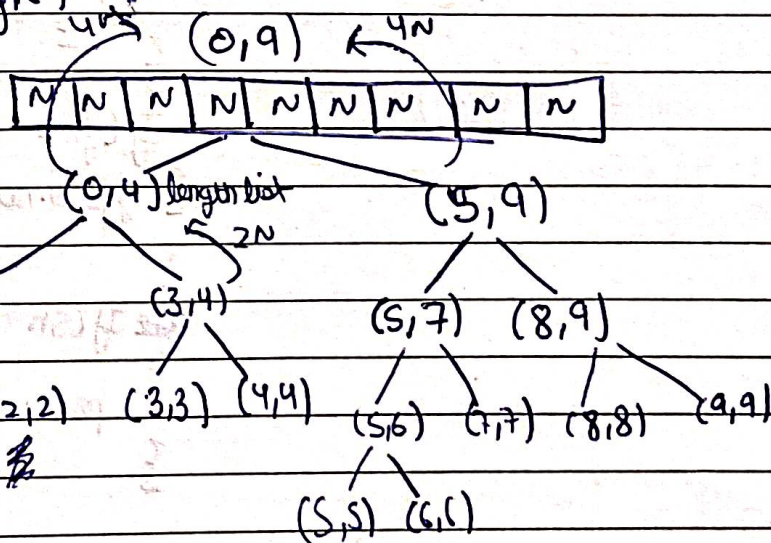
```
{
    return arr[si];
```

```
{
    int mid = (si + ei) / 2;
```

```
Node leftmerge = helper(arr, si, mid);
```

```
Node rightmerge = helper(arr, mid+1, ei);
```

```
return merge(leftmerge, rightmerge);
```



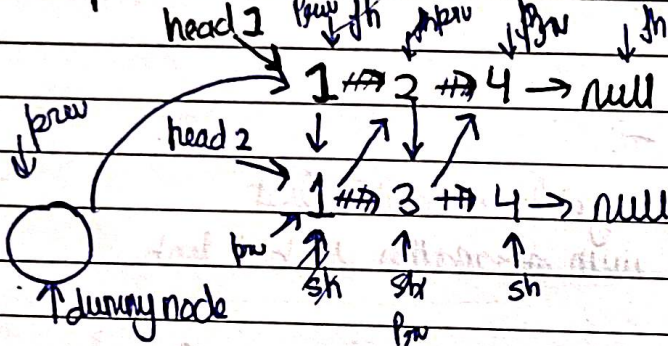
Merge two Sorted linked list

~~1 → 2 → 3~~

1 → 2 → 4

1 → 3 → 4

output: 1 → 1 → 2 → 3 → 4 → 4 → null



while (fh != null && sh != null)

{ if (fh.val < sh.val)

{ prev.next = fh

prev = prev.next

fh = fh.next

else {

prev.next = sh

prev = prev.next

sh = sh.next

}

if (fh == null)

{ prev.next = sh

}

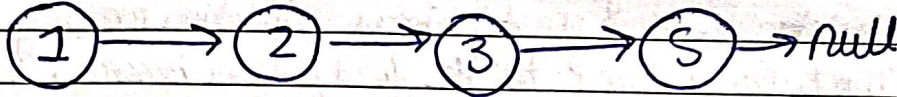
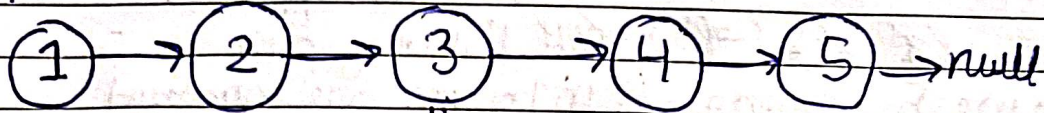
else if (sh == null)

{ prev.next = fh

}

Remove N^{th} Node from End of List

Example 1:-



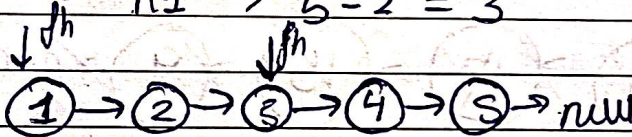
Input :- [1 -> 2 -> 3 -> 4 -> 5 -> null], $n=2$

Output :- [1 -> 2 -> 3 -> 5 -> null]

first approach:- Get the length of the linked list
then Required Index = length of LL - $n \Rightarrow$

now for removing that particular node

$$R.I \Rightarrow 5 - 2 = 3$$



int $i=0$;

while ($i \leq R.I$)

{ head $jh = jh \rightarrow next$; $i++$;

}

If ($jh \rightarrow next \neq null$) Node remove = $jh \rightarrow next$ new Node (12)

node fast = new Node (11);

If ($jh \rightarrow next \neq null$)

remove = $jh \rightarrow next$

If ($jh \rightarrow next \rightarrow next \neq null$)

{ fast = $jh \rightarrow next \rightarrow next$;

}

remove $\rightarrow next = null$;

$jh \rightarrow next = fast$;

for 1 length and $n=1$

[1], $n=1$

~~If~~ $R.I = 1 - 1 = 0$

If ($R.I = 0$)

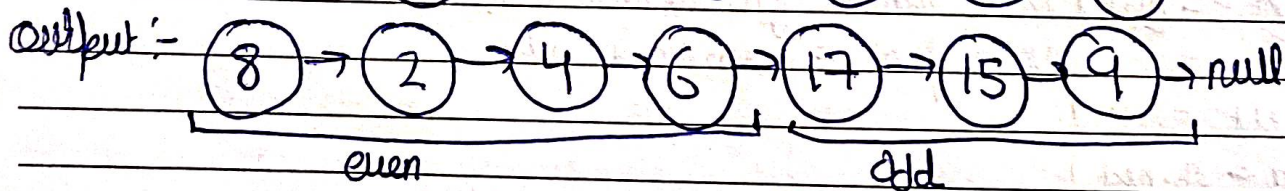
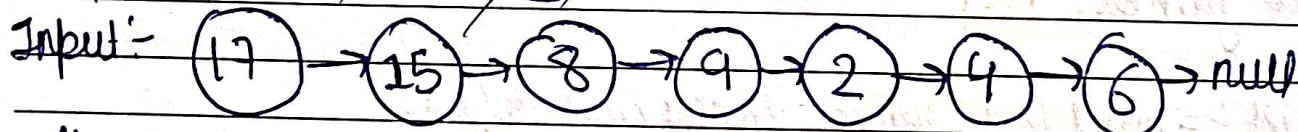
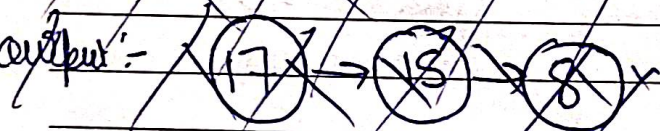
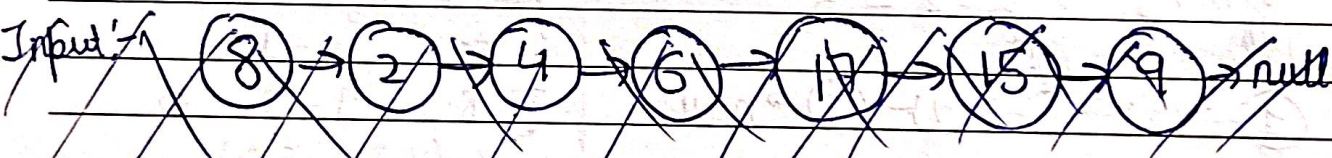
return null.

Approach - 2

with this approach we can iterate the list only once. We need to use two pointers, both pointers are initialized to head. Traverse n nodes from head with first pointer. Then the second pointer and first pointer start moving simultaneously. This keeps on doing till the first pointer becomes null. At this point the second pointer will be at the desired point.

Odd-Even Linked list

Segregate even and odd nodes in a linked list



You just need to do that for every node. Check whether this is odd or even. If it is even then link that node to the previous even chain linked list. If it is odd then link that node to the previous odd chain linked list.

odd
 OP = null, EP = null, Jh = head

```

while (Jh != null)
{
  If (even)
  {
    If (ei == null)
    { ei = Jh }
    If (ep != null)
    { ep.next = Jh;
      ep = Jh;
    }
    else { ep = Jh;
  }

```

```

}
else { (If odd)
  If (oi == null)
  { oi = Jh;
  }
  If (OP != null)
  { OP.next = Jh;
    OP = Jh;
  }
  else { OP = Jh;
}
Jh = Jh.next;
}

```

eg edge case:-

1) If the all the no are even

2) If all the no are odd.

```

If (oi == null || ei == null)
{
  return head;
}

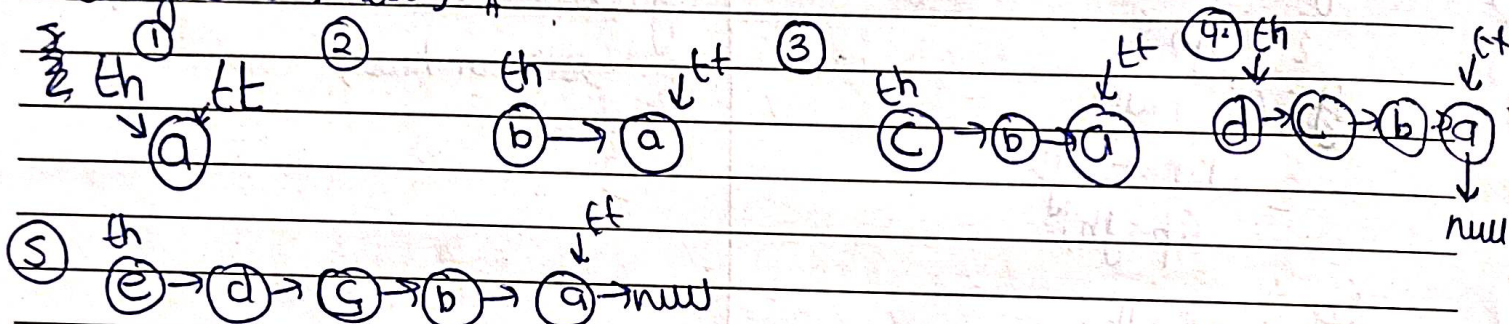
```

Reverse a linked list Using Add first method



We will make add first function where we pass it as node

add first (Node)



Node $th = \text{null};$

Node $ft = \text{null};$

TC :- $O(N)$

SP :- $O(1)$

fun add first (Node node)

{ if ($th == \text{null}$)

{ $th = \text{node};$

$ft = \text{node};$

}

$\text{node.next} = th;$

$th = \text{node};$

}

fun reverse (Node head)

{ Node $curr = \text{head};$

while ($curr != \text{null}$)

{ Node $fn = curr.next;$

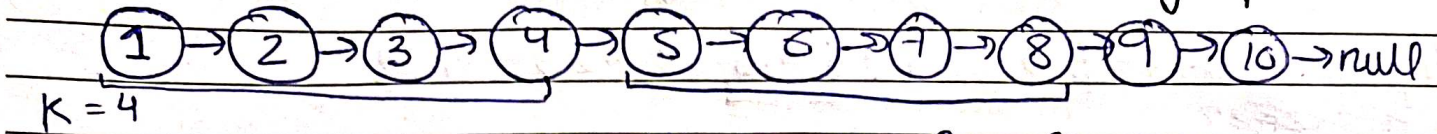
$curr.next = \text{null}$

add first ($curr$);

$curr = fn;$

}

Reverse a linked list in K groups

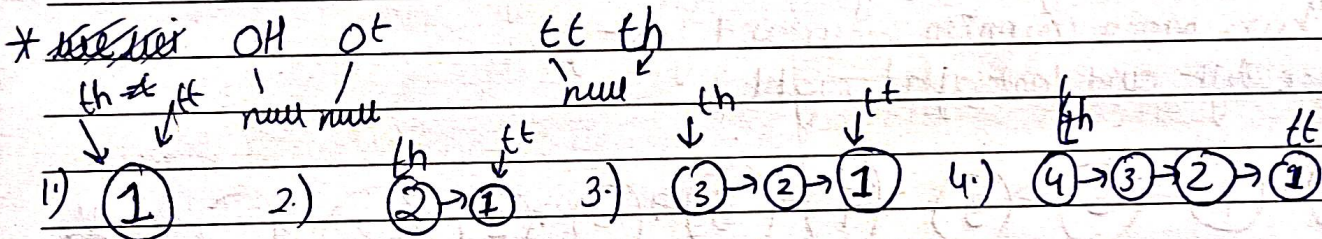


Output: $4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 8 \rightarrow 7 \rightarrow 6 \rightarrow 5 \rightarrow 9 \rightarrow 10 \rightarrow \text{null}$

Here we use add first technique to reverse the list

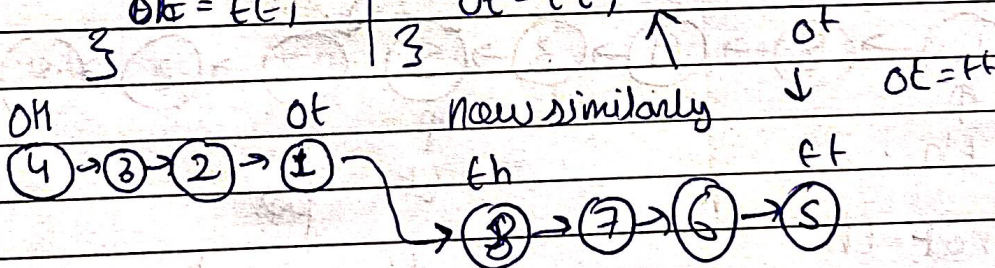
* first we find out the length of LL

* then we take four pointers tt, th, OH, Ot they all are null at initialization.



4) now $K = 4$

if $th == \text{null}$	else {
$OH = th$	$Ot \cdot \text{next} = th$
$Ot = tt$	$Ot = tt$
}	}



Now if the rest length is not multiple of K we simply do $Ot = \text{curr}$.

TC: $O(N)$

SP: $O(1)$

Reverse In Range Inplace

Input:- $1 \rightarrow 7 \rightarrow 5 \rightarrow 3 \rightarrow 9 \rightarrow 8 \rightarrow 10 \rightarrow 2 \rightarrow 2 \rightarrow 5 \rightarrow \text{null}$

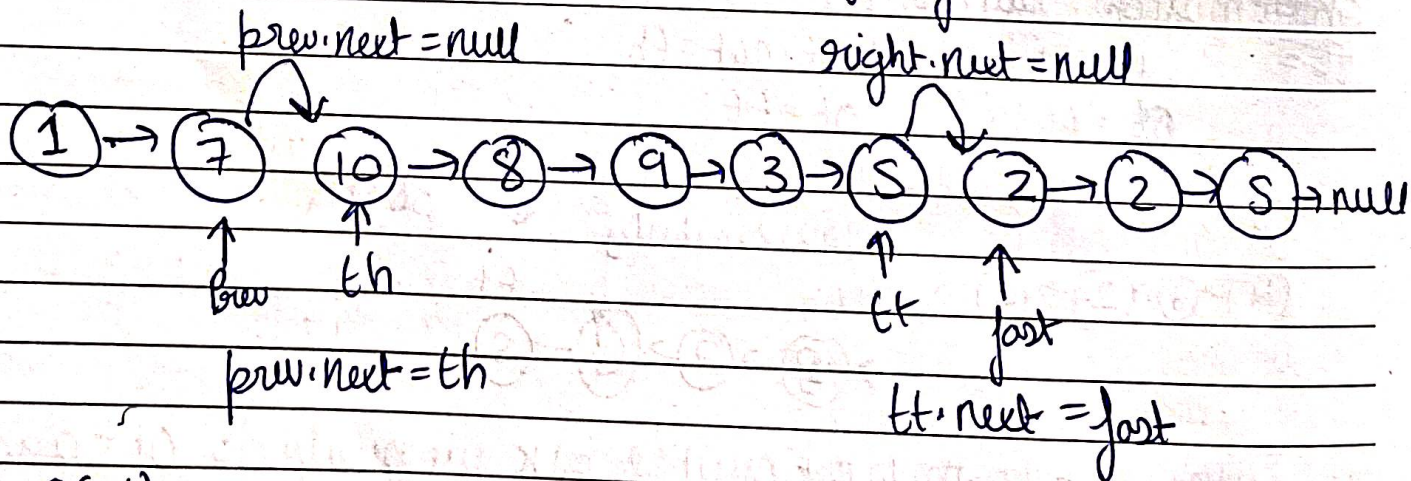
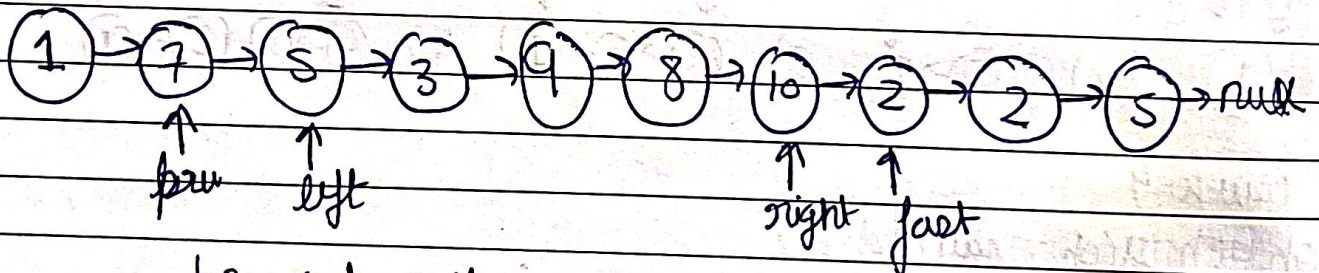
~~1 2 3 4 5 6 7 8~~

~~L=2, R=8~~ $L=3, R=7$

$1 \rightarrow 7 \rightarrow 10 \rightarrow 8 \rightarrow 9 \rightarrow 3 \rightarrow 5 \rightarrow 2 \rightarrow 2 \rightarrow 5 \rightarrow \text{null}$

One approach can be that
you can do this with (add first.) reverse strategy

using addfirst method you will get th and tt of a reverse list
just take two more counter prev and fast &
prev before left and fast after right.



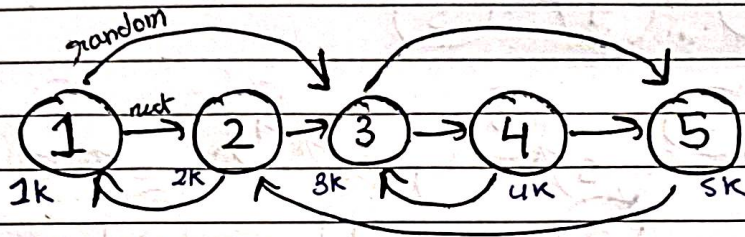
T.C :- $O(N)$

S.C :- $O(1)$

Clone a linked list with Next and Random pointer

You are given a linked list having two pointers next and random. you have to make deep copy of a linked list

Input:-



Hashmap

~~1K → 21K~~ 1K → 21K

2K → 22K

3K → 23K

4K → 24K

5K → 25K

Method 1:-



T.C :- $O(N)$, S.P :- $O(N)$

Create a ~~the~~ new linked list with same data and ~~use~~ for random pointer use hashmap to store and then manage the random pointer

```
HashMap<node, node> mp = new HashMap<>();
```

```
Node curr = head;
```

```
Node nhead = new Node(-1);
```

```
Node prev = nhead;
```

```
while (curr != null)
```

```
{ Node node = new Node(curr.data);
```

```
prev.next = node;
```

```
mp.put(curr, node);
```

```
curr = curr.next;
```

```
prev = prev.next;
```

```
}
```

```
nhead = nhead.next;
```

```
curr = nhead;
```

```
Node c1 = head;
```

```
Node c2 = head;
```

```
while (c1 != null)
```

```
{ c2.random = c1.random != null ?  
mp.get(c1.random):  
null;
```

```
c1 = c1.next;
```

```
c2 = c2.next;
```

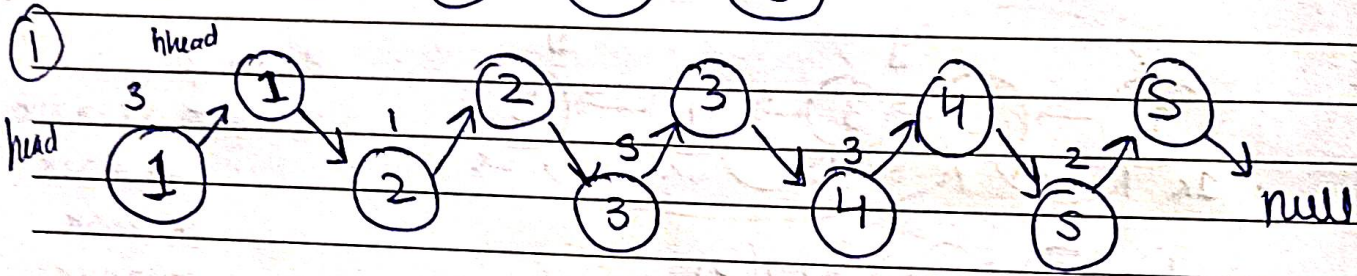
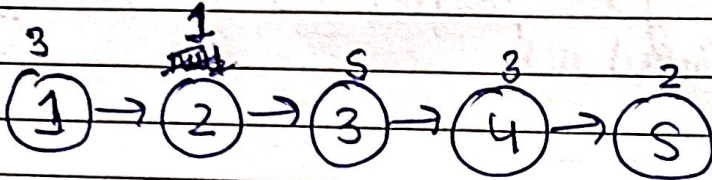
```
}
```

```
return nhead;
```

```
}
```

Method 2:-

$O(1)$ Space and $O(N)$ Time Complexity



② Copy Random pointer

③ extract deep copy linked list:-

```

1) void copyList(head)
{
    curr = head;
    while (curr != null)
    {
        fn = curr->next;
        Node node = new Node(curr->data);
        curr->next = node;
        node->next = fn;
    }
}

2) void copyRandomPointer(Node head)
{
    curr = head;
    while (curr != null)
    {
        Node random = curr->random;
        if (random != null)
            curr->next->random = random->next;
        curr = curr->next->next;
    }
}
  
```

```

3) Extract deep copy LL
Node deepCopyLL(Node head)
Node dummy = new Node(-1);
Node prev = dummy;
curr = head;
while (curr != null)
{
    prev->next = curr->next;
    curr->next = curr->next->next;
    prev = prev->next;
    curr = curr->next;
}
return dummy->next;
}
  
```

Multiplication of two linked list

(2) → (4) → (5) → (1) → (9) → null list 1

(4) → (7) → (6) → null list 2

Output: (1) → (1) → (6) → (7) → (1) → (0) → (4) → (4) → null

How:-

1. Reverse both linked list

(9) → (1) → (5) → (4) → (2) → null, (6) → (7) → (4) → null

2. Put iterator on second list and multiple one by one

2.1 → (9) → (1) → (5) → (4) → (2) → null, (6)

null ← (54) ← (9) ← (2) ← (5) ← (2)

Similarly we get two more linked list from (7) and (4)
ie..

Now we have to add the linked list

3. we will specify the counter

null ← (5) ← (4) ← (9) ← (2) ← (5) ← (2)

1. add with dummy list null ← (5) ← (4) ← (9) ← (2) ← (5) ← (2) ← (-1)

Counter

dummy

code:-

```

public static multiplytwoll(ListNode L1, ListNode L2)
{
    L1 = reverse(L1);
    L2 = reverse(L2);
    ListNode l2_itr = L2;
    ListNode dummy = new ListNode(-1);
    ListNode ans_itr = dummy;
    while (l2_itr != null)
    {
        ListNode prod = multiplytwollwithdigit(L1, l2_itr.val);
        l2_itr = l2_itr.next;
        addtwolinkedlist(prod, ans_itr);
        ans_itr = ans_itr.next;
    }
    return reverse(dummy.next);
}

```

```

public static ListNode multiplytwollwithdigit(ListNode headl1, int dig)
{
    ListNode dummy = new ListNode(-1);
    ListNode ac = dummy;
    ListNode cur = head;
    int carry = 0;
    while (cur != null || carry != 0)
    {
        int sum = (carry + (cur != null ? cur.val * dig : 0));
        int digit = sum % 10;
        carry = sum / 10;
        ac.next = new ListNode(digit);
        if (cur != null) cur = cur.next;
        ac = ac.next;
    }
    return dummy.next;
}

```

```
public static void addTwoLinkedList(ListNode head, ListNode another) {
```

```
    ListNode C1 = head; // always greater
```

```
    ListNode C2 = another;
```

```
    int carry = 0;
```

```
    while (C1 != null || carry != 0)
```

```
    { int Sum = carry + (C1 != null ? C1.val : 0) + (C2 != null ? C2.val : 0);
```

```
        int digit = Sum % 10;
```

```
        carry = Sum / 10;
```

```
        if (C2 != null) C2.val = digit;
```

```
        else C2 = new ListNode(digit);
```

```
        if (C1 != null) C1 = C1.next;
```

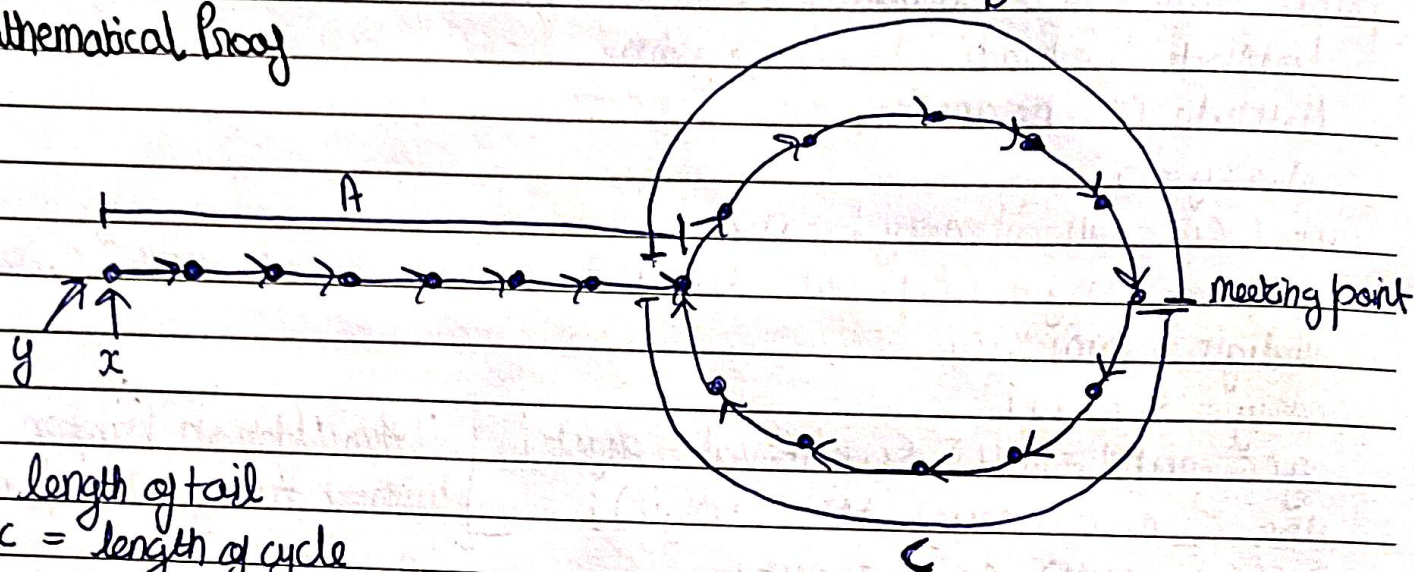
```
        C2 = C2.next;
```

```
    }
```

// this addition happen
without taking extra space.

Detection of cycle in a linkedlist

Mathematical Proof



A = length of tail

$B+c$ = length of cycle

n = No. of rotation taken by X before meeting point.

m = No. of rotation taken by Y before meeting point.

$$dx = A + (B+c)n + B \quad \text{(Total x travel distance)} \quad - (1)$$

Last distance move to meet at point.

$$dy = A + (B+c)m + B \quad - (2)$$

We know that.

~~distance~~ speed = distance / time. , suppose they meet at time $t = T$

$$x = \frac{dx}{T}, \quad y = \frac{dy}{T}$$

$$\text{on } T = \frac{dx}{x}, \quad T = \frac{dy}{y}$$

$$\Rightarrow \frac{dx}{x} = \frac{dy}{y} \Rightarrow \left(\frac{y}{x}\right) dx = dy$$

$$\text{let } (y/x) = g$$

$$g dx = dy \quad - (a)$$

from (1) and (2) put in (a)

$$g(A+B) + gn(B+c) = (A+B) + m(B+c)$$

$$(A+B)(g-1) = (B+c)(m-gn)$$

$$(A+B) = \frac{(B+1)(m-xr)}{r-1}$$

from this formula we can conclude that if $r-1 \rightarrow 0$, $\therefore$

$$(B+1)(m-xr) (A+B) \rightarrow \infty, \quad \exists A+B = \text{length of cycle}$$

$\therefore$ we can say that $r-1 > 0$

$$r > 1$$

$$r \in \mathbb{R}^+, r \neq 1$$

* what if $r=1$

$$\therefore \frac{y}{x} = 1$$

$$y = x$$

Here ~~the~~ both pointers will move on a same speed

$\therefore$ every point became meeting point.

~~Hence this will be in~~

* we can take any ratio of but it should be greater than 1

* But if we can move with any ratio then why we always take a difference of 2 between slow and fast pointer.

It is because, if we move pointers other than ratio of 2.

They will meet forever but it

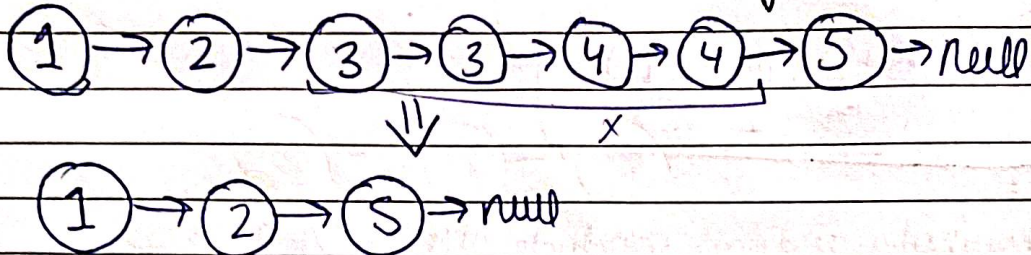
takes time. Therefore the ~~opti~~ optimum approach would be move fast pointer by 2 and slow pointer by 1.

* Another approach this question can also be done with using Hashset(), store every Node ~~value~~ in Hashset if particular

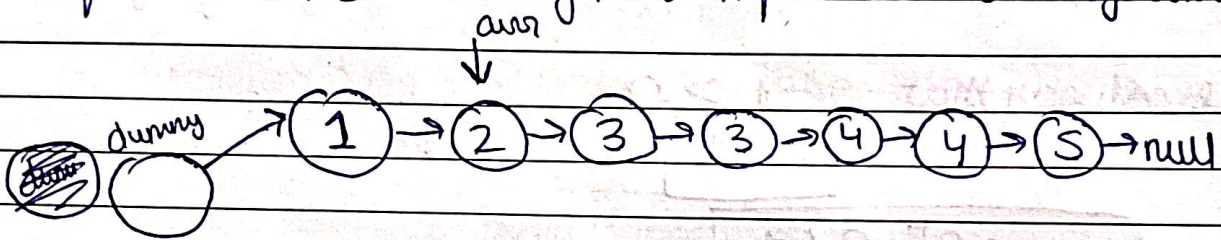
Node visit again that means Hashset also contains it.

$\therefore$ you can detect cycle in a LL.

Remove all Occurrence of duplicates in a LL



This question can be done by Maintain previous dummy iteration



```
Node dummy = new Node(-1);
```

```
Node iter = dummy;
```

```
dummy.next = head;
```

```
Node curr = head.next;
```

```
while (curr != null)
```

```
{ if (iter.next.data == curr.data)
```

```
{ Boolean isLoopRun = false;
```

```
while (curr != null && iter.next.data == curr.data)
```

```
{ curr = curr.next;
```

```
isLoopRun = true;
```

```
}
```

```
if (isLoopRun)
```

```
{ iter.next = curr;
```

```
}
```

```
else
```

```
temp = temp.next;
```

```
if (curr != null) curr = curr.next;
```

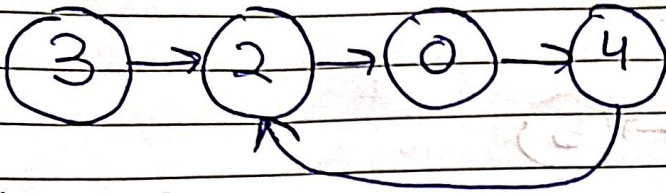
```
}
```

```
return dummy.next;
```

We touch

Similar:- Remove a loop in a linked list.

Detect first Node of a linked list



Output:- 2

Approach

~~Using Hashset~~

Store every node in a hashset and also check if the particular node is visited earlier or not if yes just return that node it is your starting point.

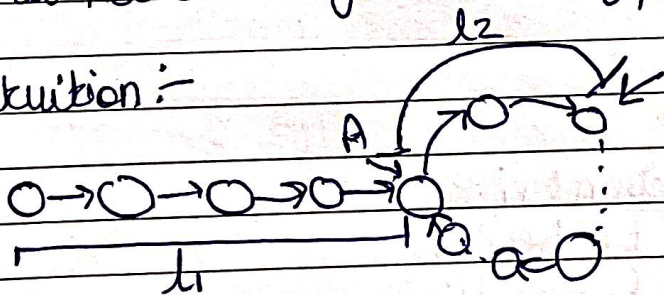
TC :- $O(n)$

Sp :- $O(n)$

Optimise solution:-

1. Find the meeting point where slow and fast pointers meet
2. Then after put entry as a new pointer to the head and move slow and enter pointer simultaneously. When they meet return that node. it is your starting point.

Intuition:-



Suppose this is our meeting point.

C = length of the cycle of LL

S = Slow pointer, f = fast pointer

$$S = L_1 + L_2$$

$$f = (L_1 + L_2) + nC$$

n = no. of extra

turn that fast pointer takes before collision

we know that fast point travel twice distance than slow pointer

$$2(L_1 + L_2) = L_1 + L_2 + nC$$

$$L_1 + L_2 = nC$$

$$L_1 = nC - L_2$$

at A (entry point)

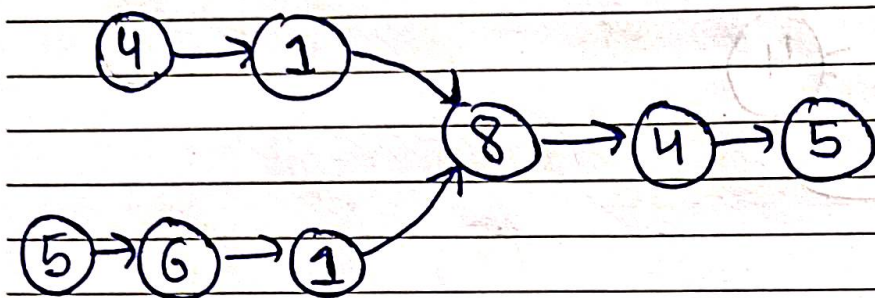
$$nC - L_2 \Rightarrow$$

Therefore we start from the beginning

We touch your electricity everyday!

they bound to meet at entry point

Intersection of two linked list

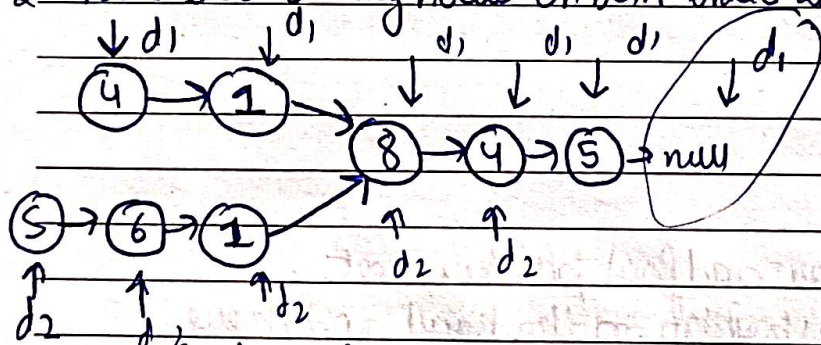


first Approach

1. find the difference between two linked list and move the ~~same~~ bigger LL by difference, then move simultaneously the pointer that is

Second Approach

2. Take two dummy nodes on both linked list then move ~~to~~ both of them



whenever d1 reaches null of d2 reaches null assign it other LL.

Pseudo code :- If (head1 == null || head2 == null) return null;

t1 = head1, t2 = head2;

while (t1 != t2)

{ t1 = t1.next;

t2 = t2.next;

If (t1 == t2) return t1;

If (t1 == Null) t1 = head2;

If (t2 == Null) t2 = head1;

return t1;